



Mash-Up
iOS Team





사랑의 기술



2026.04.14(Sat)





사랑의 기술 in iOS



2026.04.14(Sat)





Index

1 사랑의 기술

2 사랑과 iOS의 공통점

3 사랑과 iOS의 차이점

4 사랑, 해야하는가?

5 실습하기

6 결혼하거나 결별하거나

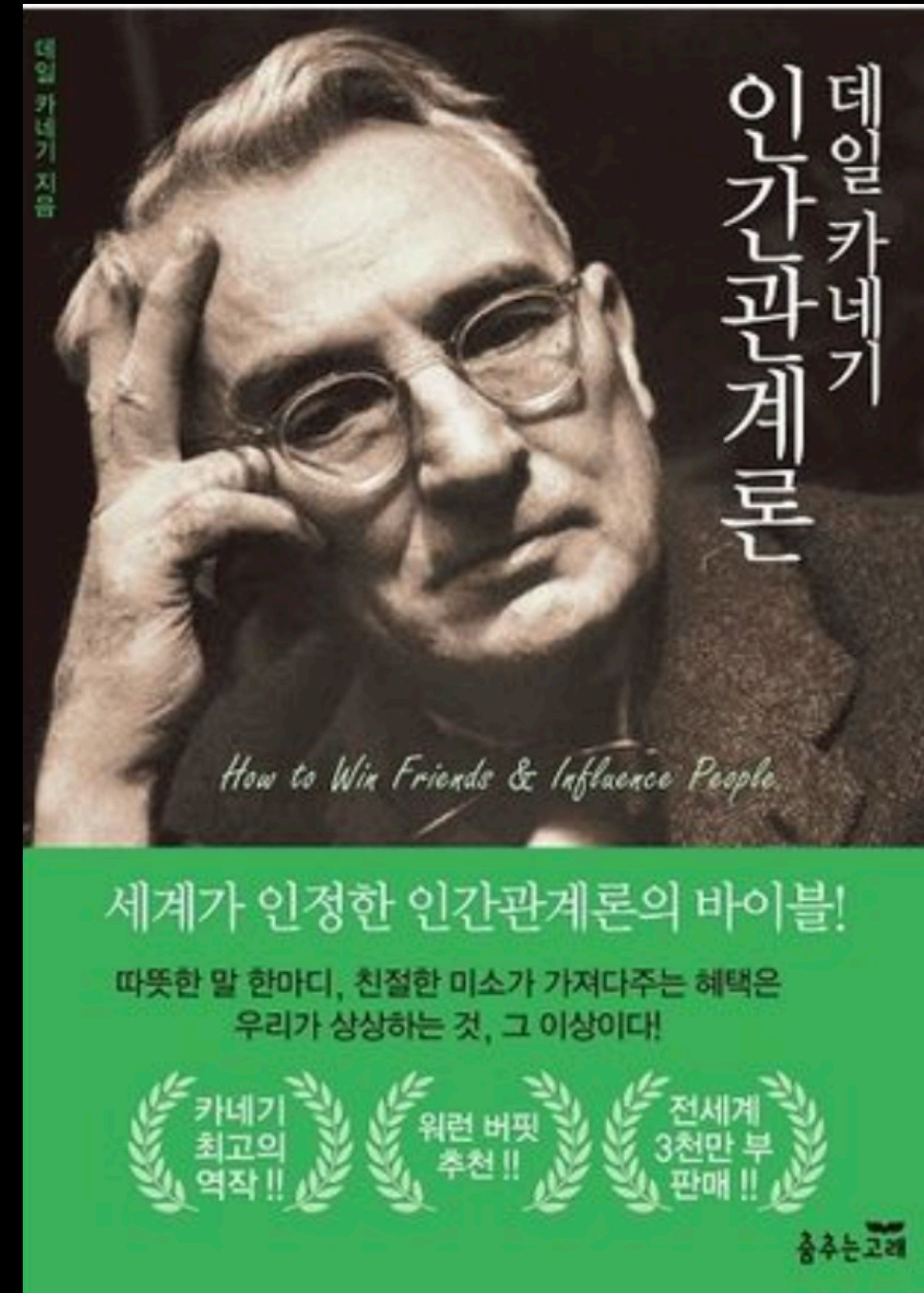
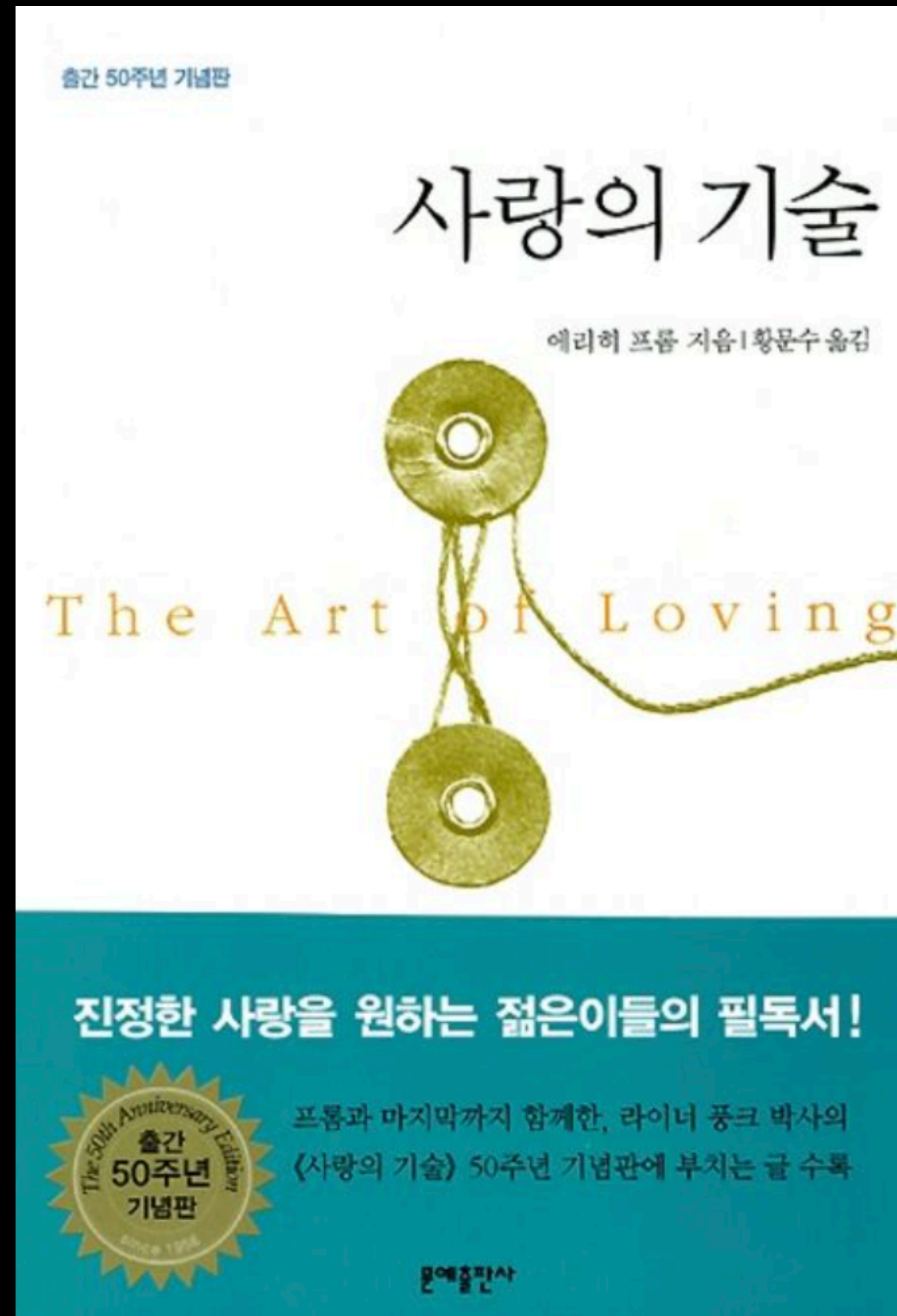


Mash-Up
iOS Team

사랑의 기술



Mash-Up
iOS Team





Mash-Up
iOS Team

안녕 지피티야 너는 사랑이 뭐라고 생각해?



안녕 지피티야 너는 사랑이 뭐라고 생각해?

사랑을 한 문장으로 정의하긴 어렵지만, 나는 이렇게 봐.

사랑이란

상대의 존재와 성장을 진심으로 바라며, 그에 맞게 행동하려는 지속적인 의지야.

감정만으로는 부족해.

설렘은 시작일 수 있지만, 사랑의 본질은 태도와 선택에 더 가깝다고 생각해.

구성 요소를 나눠보면

- 1. 관심
상대를 알고 싶어 하고, 실제로 이해하려는 노력.
- 2. 존중
내 기준이 아니라 상대의 기준과 경계를 인정하는 것.
- 3. 책임
좋을 때만이 아니라 어려울 때도 관계를 유지하려는 자세.
- 4. 행동
말이 아니라 시간, 에너지, 선택으로 증명되는 것.

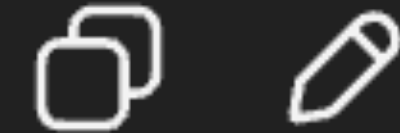
사랑 vs. 비슷하지만 다른 것

구분	특징
설렘	강렬하지만 짧을 수 있음
집착	‘내 감정’ 중심, 통제하려 함
의존	혼자서는 불안해서 붙잡음
사랑	‘우리’의 (↓) 성장을 함께 고려



Mash-Up
iOS Team

그러면 사랑하기 위해서는 뭘 해야할까???



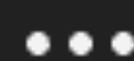


Mash-Up
iOS Team

그러면 사랑하기 위해서는 뭘 해야할까???



플러팅.

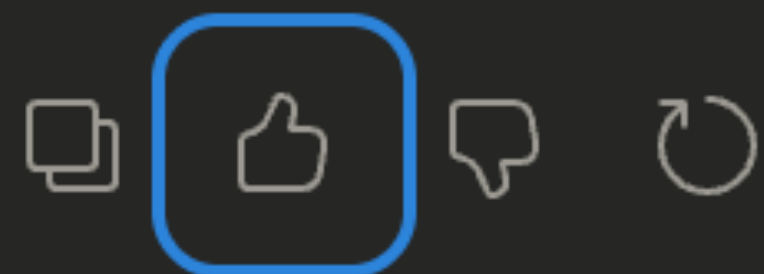




Mash-Up
iOS Team

그러면 이제 플러터와 플러팅은 한 껏 차이니까 똑같은거네??

네 맞습니다 플러터와 플러팅은 같다고 볼 수 있습니다.



긍정적인 피드백 제공



About Flutter



2026.04.14(Sat)





Index

1 플러터의 기술

2 플러터와 iOS의 공통점

3 플러터와 iOS의 차이점

4 플러터, 해야하는가?

5 실습하기

6 크로스플랫폼 하거나
네이티브 하거나

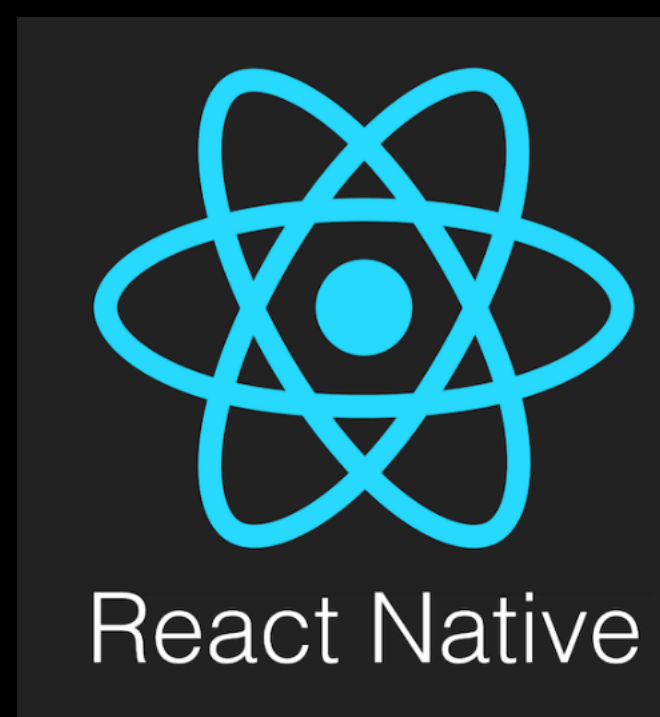


Mash-Up
iOS Team

플러터의 기술



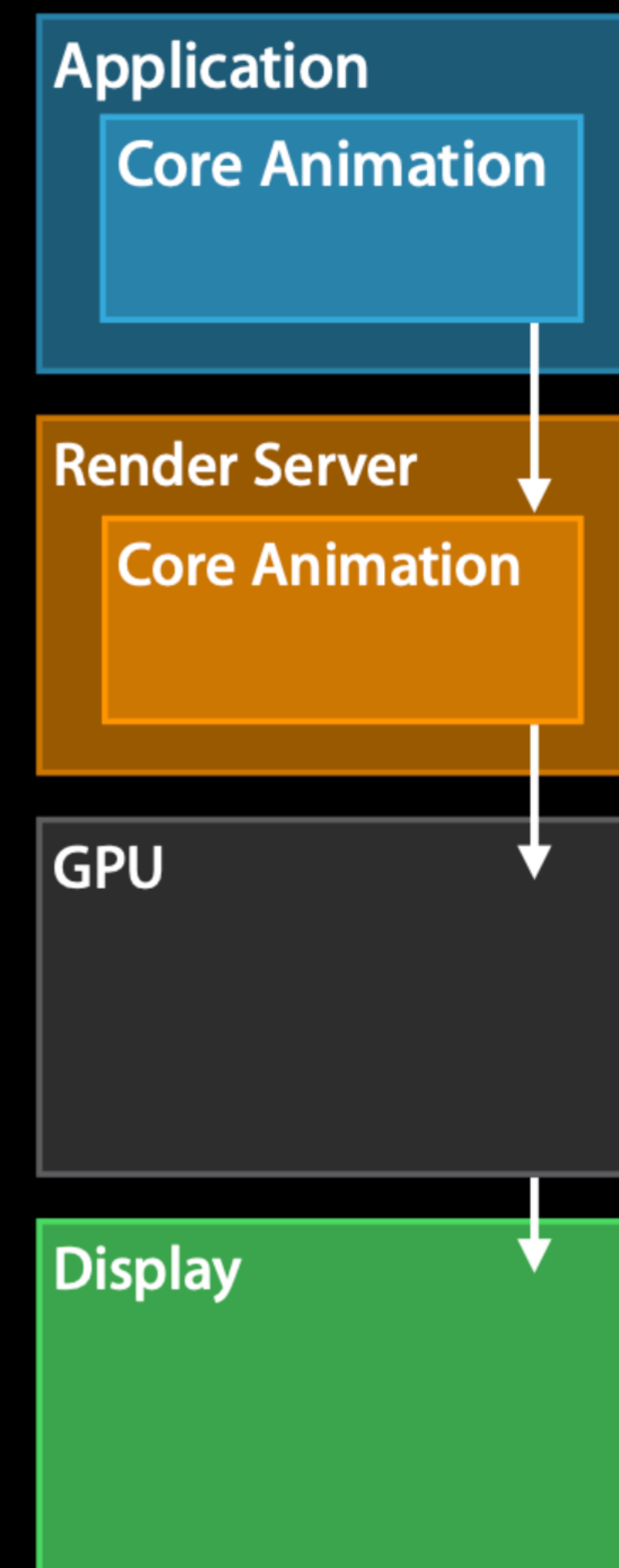
Mash-Up
iOS Team

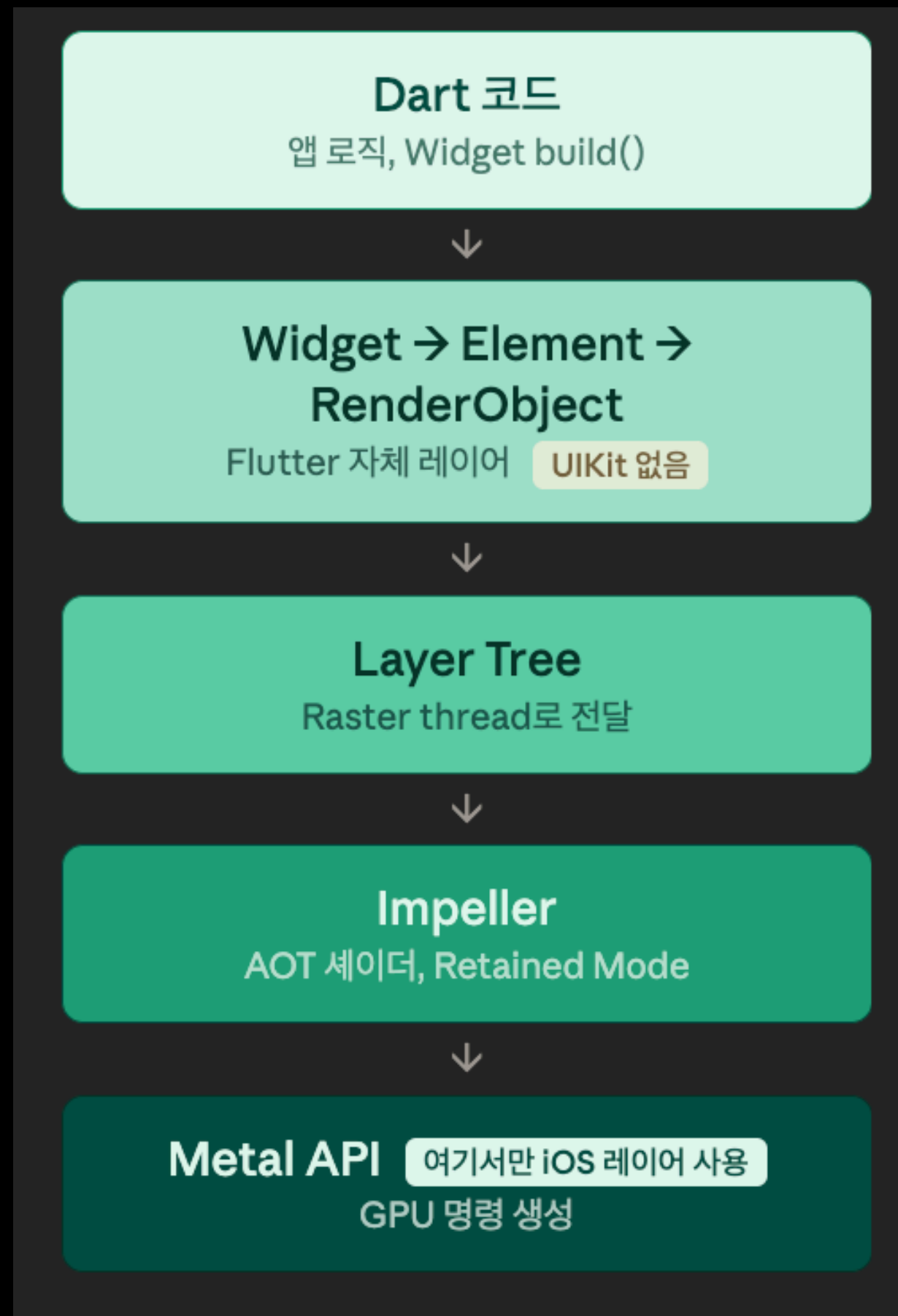




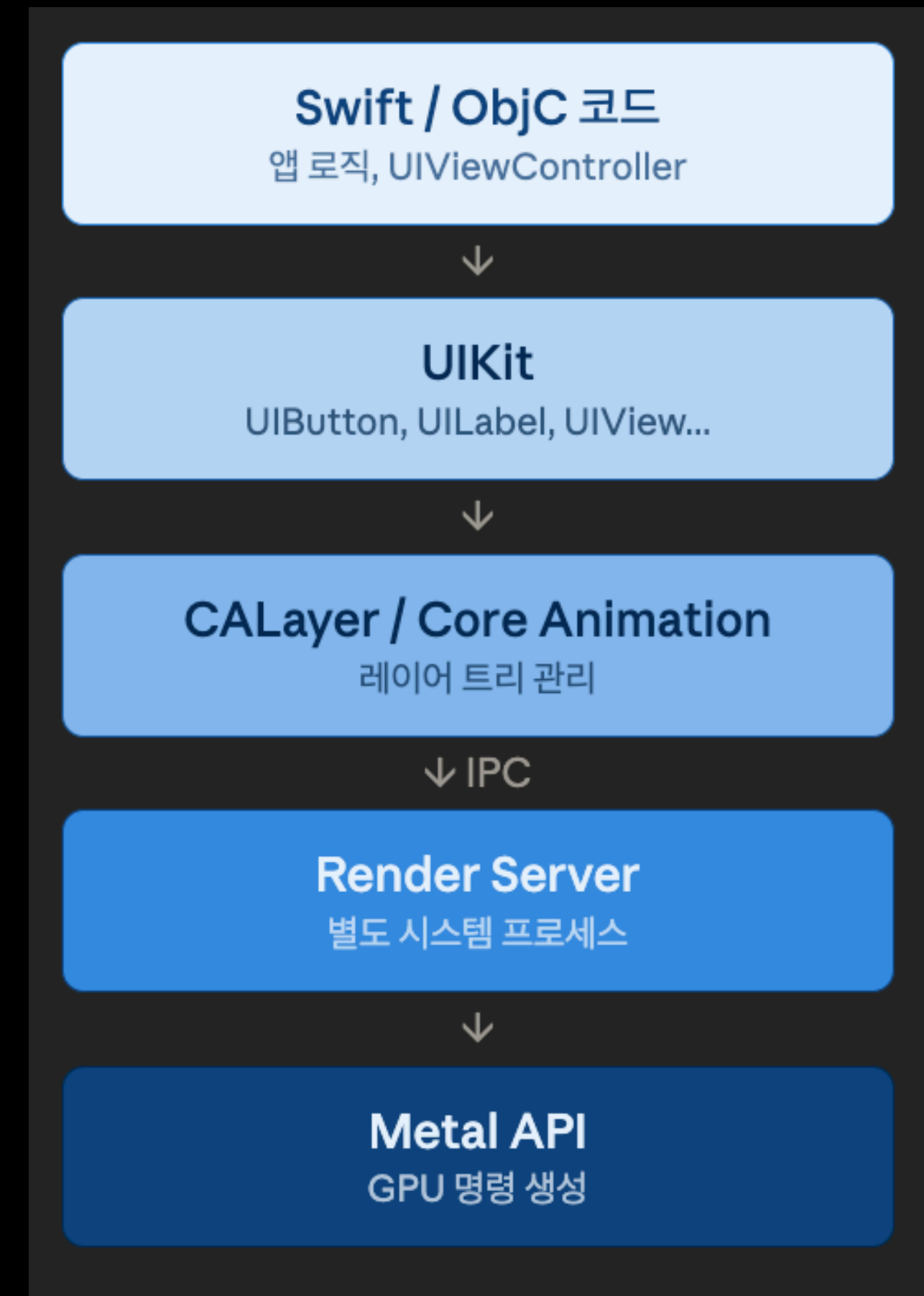
①	User input	Responses to input gestures (keyboard, touchscreen, etc.)	
②	Animation	User interface changes triggered by the tick of a timer	
③	Build	App code that creates widgets on the screen	
④	Layout	Positioning and sizing elements on the screen	RENDERING
⑤	Paint	Converting elements into a visual representation	
⑥	Composition	Overlaying visual elements in draw order	
⑦	Rasterize	Translating output into GPU render instructions	

OS한테 빈 캔버스 하나만 빌리고, 나머지는 직접 그림





Flutter



iOS





UIKit

- 명령형

```
button.setTitle("확인", for: .normal)
button.backgroundColor = .blue
label.text = "\(count)개"
```

SwiftUI

- 선언형

```
115
116     @State var count = 0
117
118     var body: some View {
119         Text("\(count)개")
120         Button("추가") { count += 1 }
121     }
122
```




Flutter

- 선언형
- 불변형임
- Widget 단위임
- StatelessWidget임
- StatefulWidget 도 있지롱

```
1  ∨ class MyLabel extends StatelessWidget {  
2      final String text;  
3      const MyLabel(this.text);  
4  
5      @override  
6  ∨  Widget build(BuildContext context) {  
7      return Text(text);  
8  }  
9  }
```



Flutter

- 선언형
- 불변형임
- Widget 단위임
- StatelessWidget임
- StatefulWidget 도 있지롱

```
11 class MyLabel extends StatelessWidget {  
12   final String text;  
13   const MyLabel(this.text);  
14  
15   @override  
16   Widget build(BuildContext context) {  
17     return Text(text);  
18   }  
19 }
```

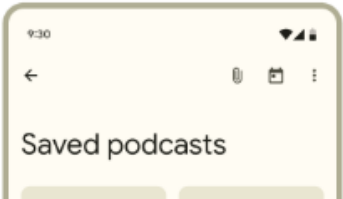
```
class _CounterState extends State<Counter> {  
  int count = 0;  
  
  @override  
  void initState() {  
    super.initState();  
  }  
  
  @override  
  void dispose() {  
    super.dispose();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        // const 없음 - count가 바뀌니까 매번 diff 필요  
        Text('$count개'),  
        // const 있음 - 절대 안 바뀌니까 Element가 diff 자체를 건너뛴  
        const Text('고정 레이블'),  
      ],  
    );  
  }  
}
```



Basic widgets

A catalog of Flutter's basic widgets. Widgets to know before building your first Flutter app.

Widgets to know before building your first Flutter app.



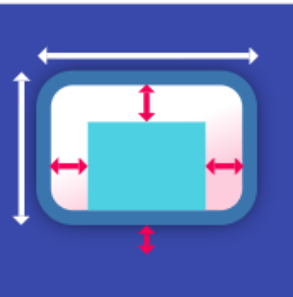
AppBar

Container that displays content and actions at the top of a screen.



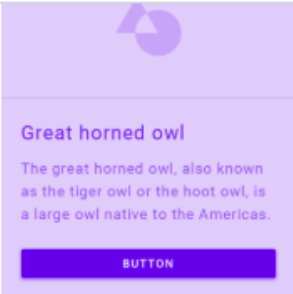
Column

Layout a list of child widgets in the vertical direction.



Container

A convenience widget that combines common painting, positioning, and sizing widgets.



ElevatedButton

A Material Design elevated button. A filled button whose material elevates when pressed.



FlutterLogo

The Flutter logo, in widget form. This widget respects the IconTheme.



Icon

A Material Design icon.



Image

A widget that displays an image.



Placeholder

A widget that draws a box that represents where other widgets will one day be added.



Row

Layout a list of child widgets in the horizontal direction.

Rendered image or visualization of the Scaffold widget.

Scaffold

Implements the basic Material Design visual layout structure. This class provides APIs for showing drawers, snack bars, and bottom sheets.

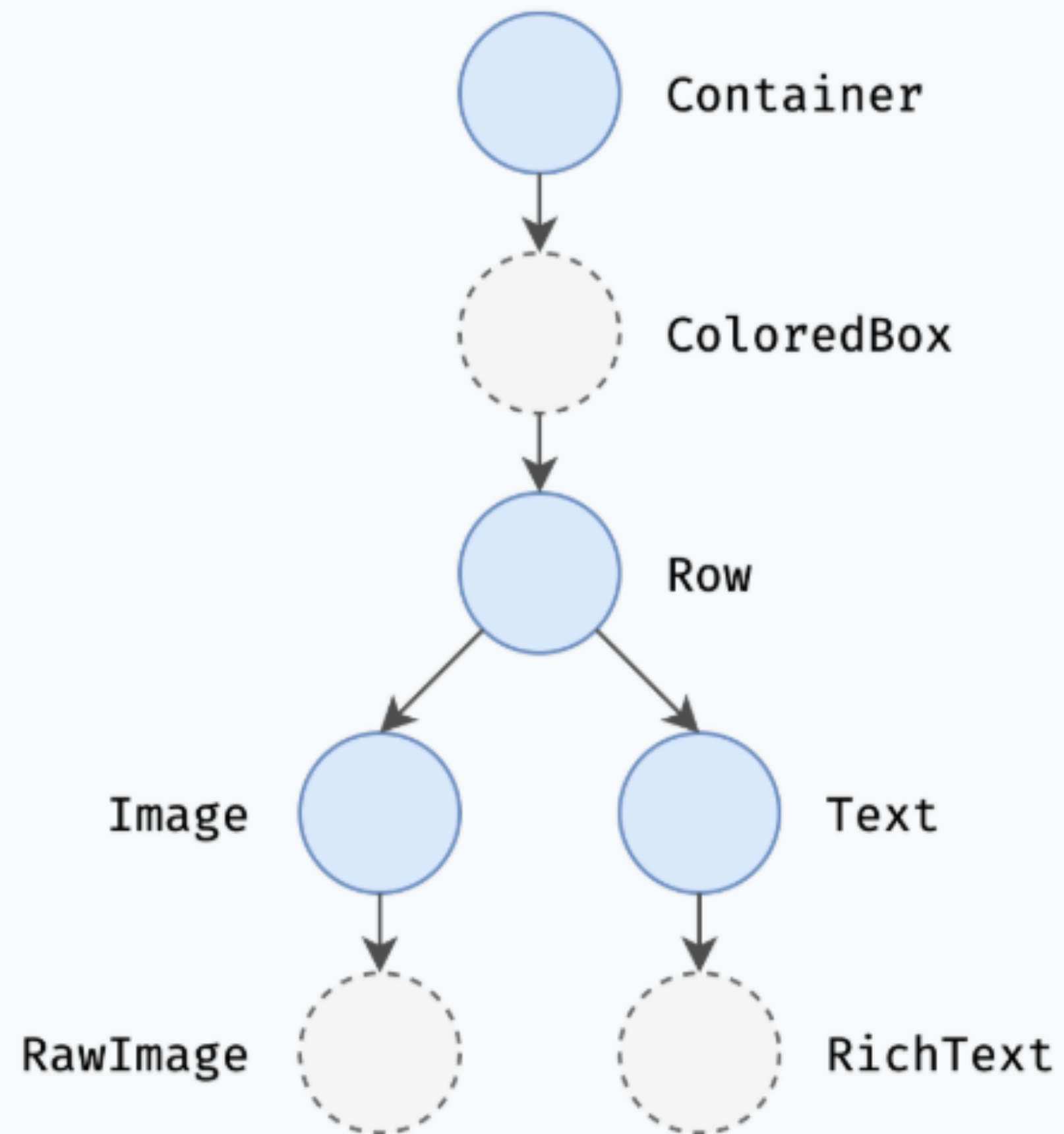


Text

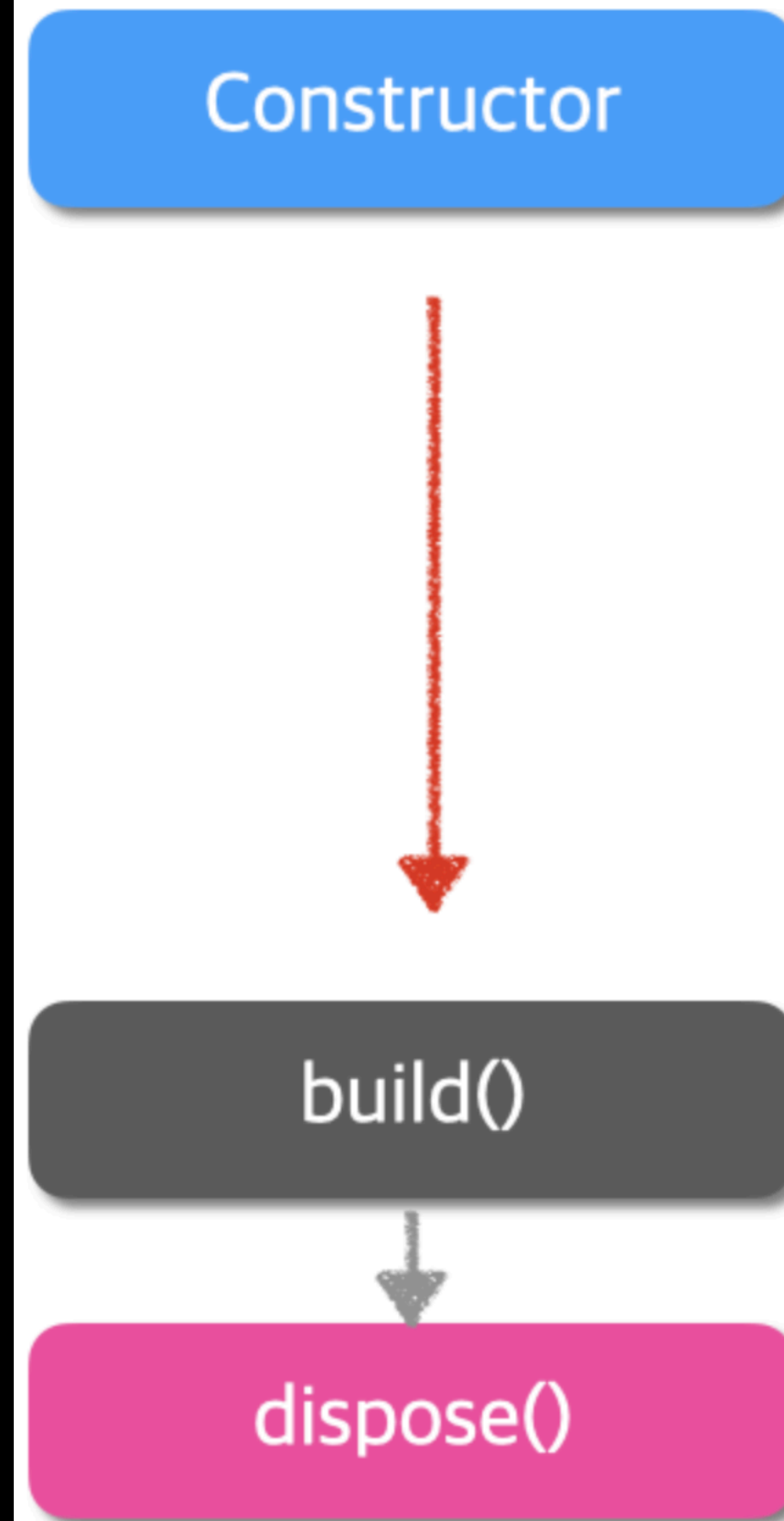
A run of text with a single style.



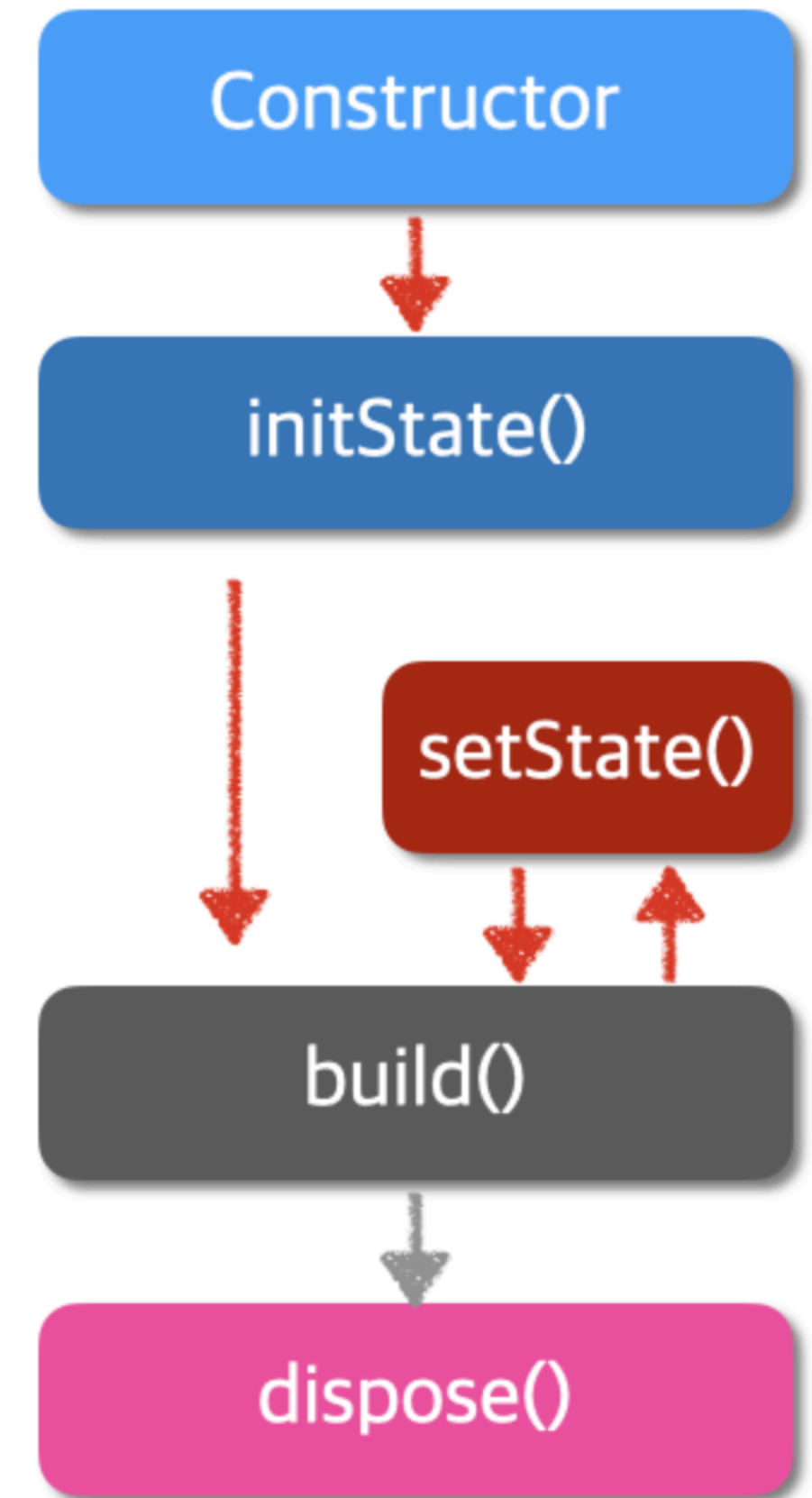
Widgets



Stateless Widget

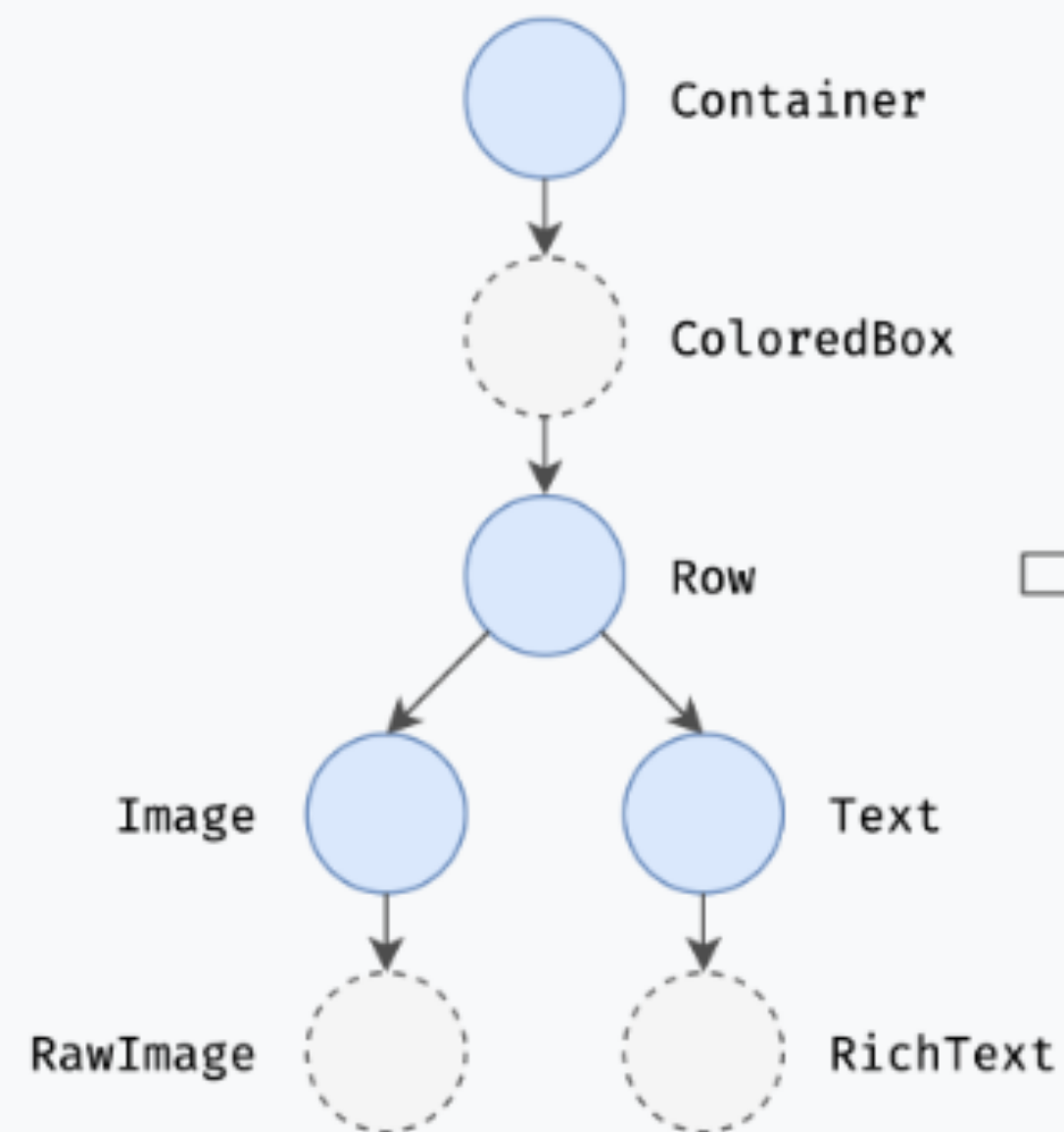


Stateful Widget

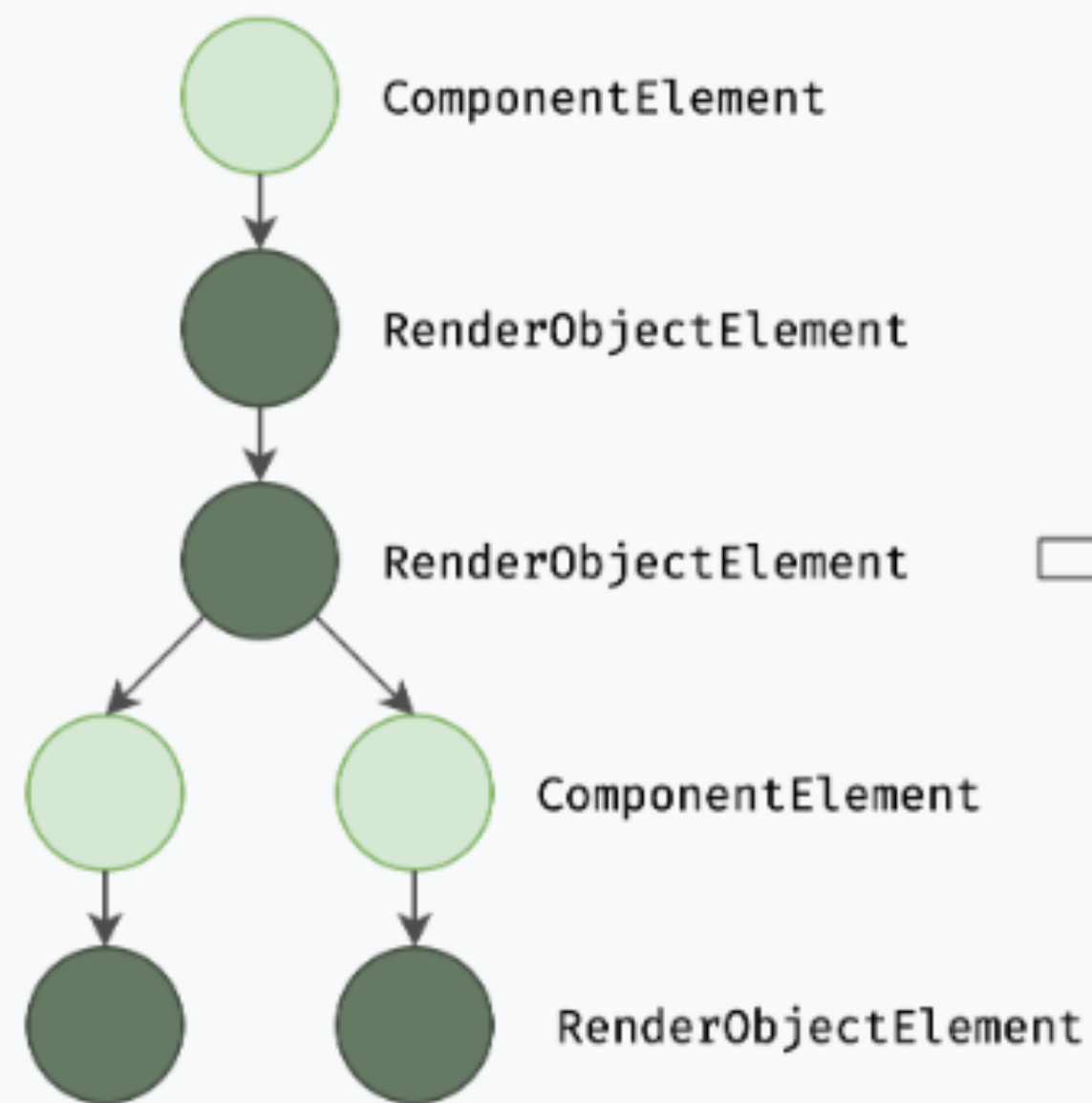




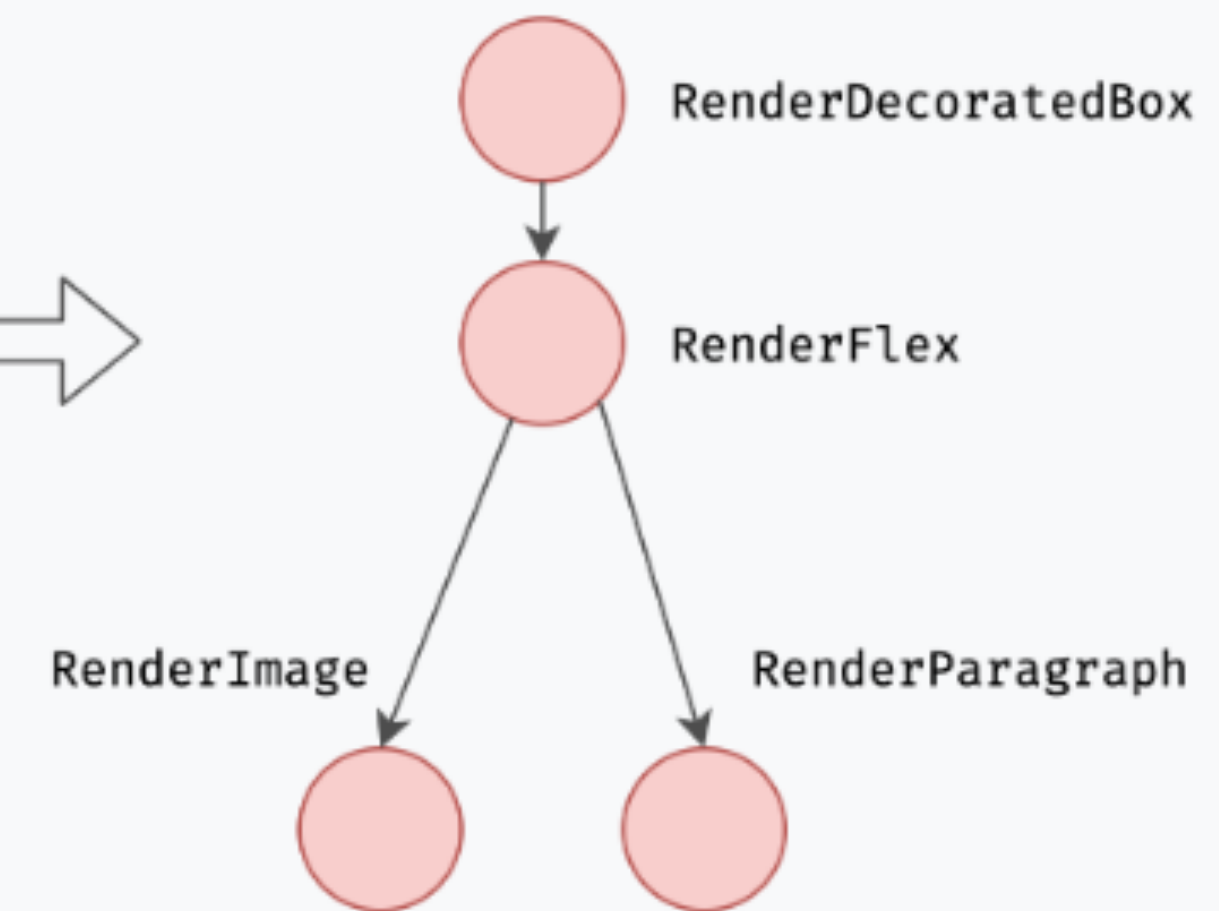
Widgets

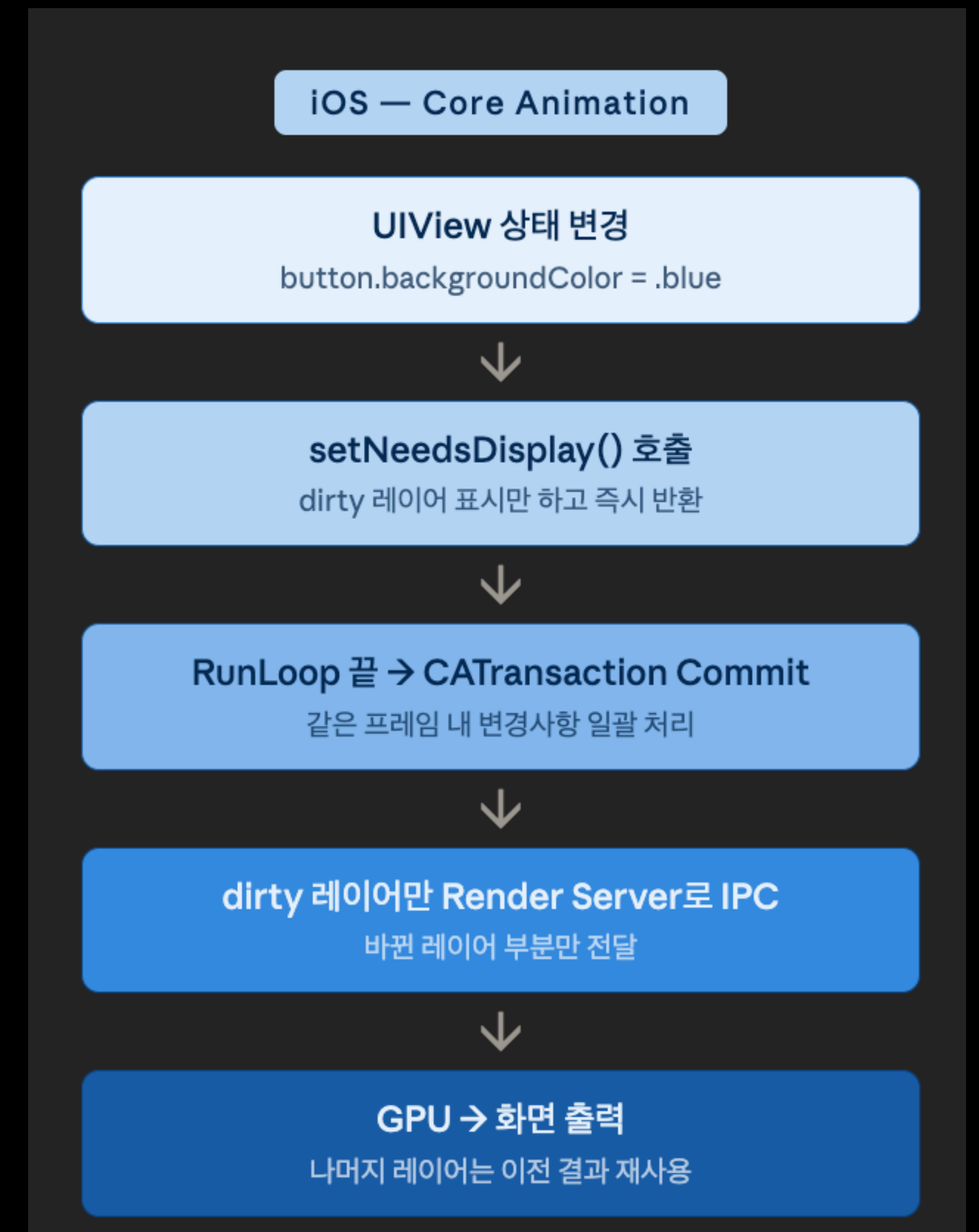
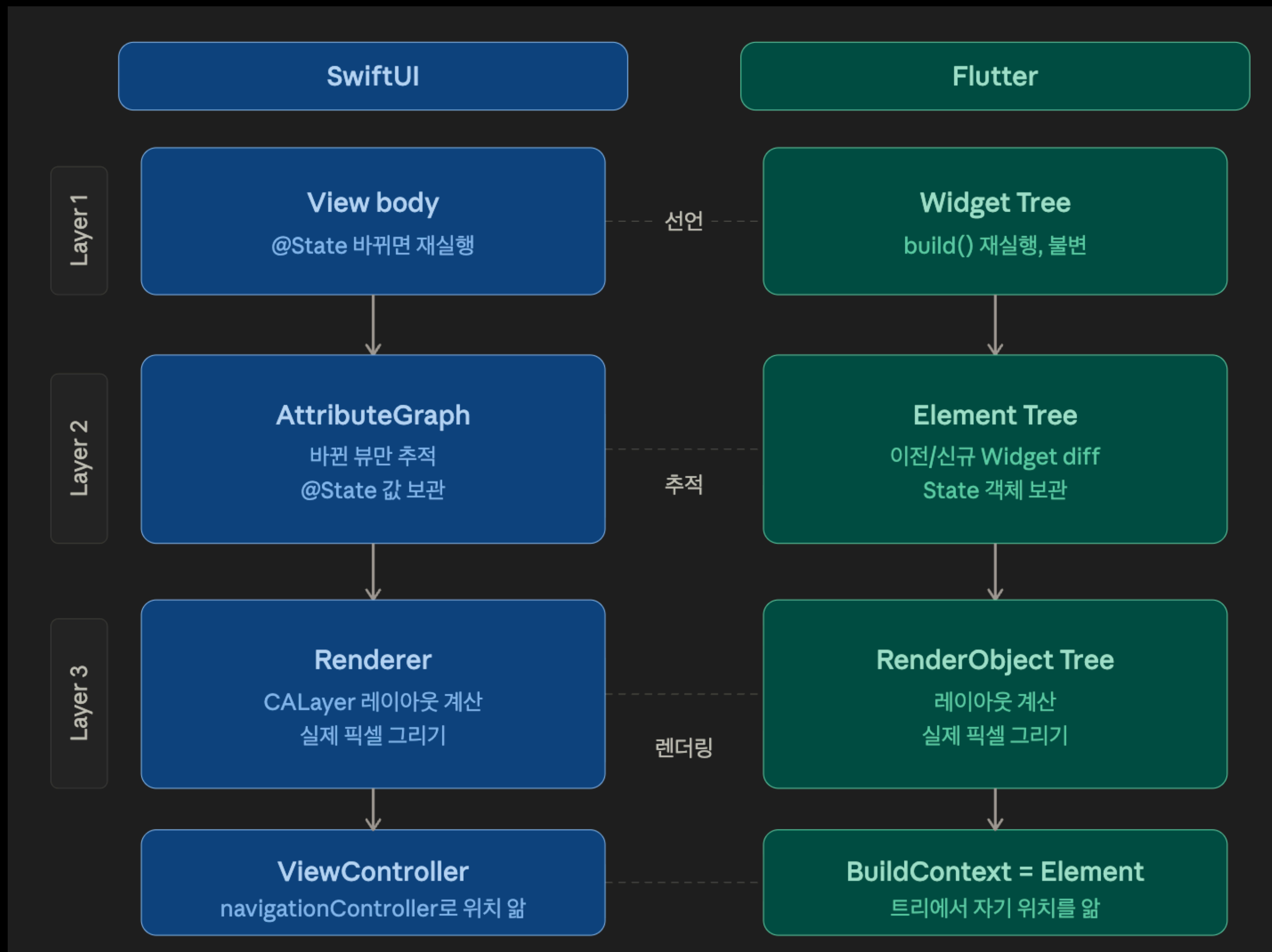


Element Tree



Render Tree







Mash-Up
iOS Team

플러터와 iOS의 메모리



Flutter is
single-threaded??

What to do
then??



플러터는 싱글 스레드입니다.
근데 비동기임ㄷㄷ



가수 비와 그의 동기들
줄이면 비동기



iOS 앱

시스템이 알아서

기타 시스템 스레드

Render Server
렌더링, GPU

URLSession 내부
I/O

개발자가 관리

메인 스레드
UIKit, 이벤트, UI 코드

GCD global queue
백그라운드 CPU 연산

Flutter 앱

엔진이 알아서 (접근 불가)

Platform thread
iOS API 호출

Raster thread
렌더링, GPU 명령

I/O thread
이미지 디코딩

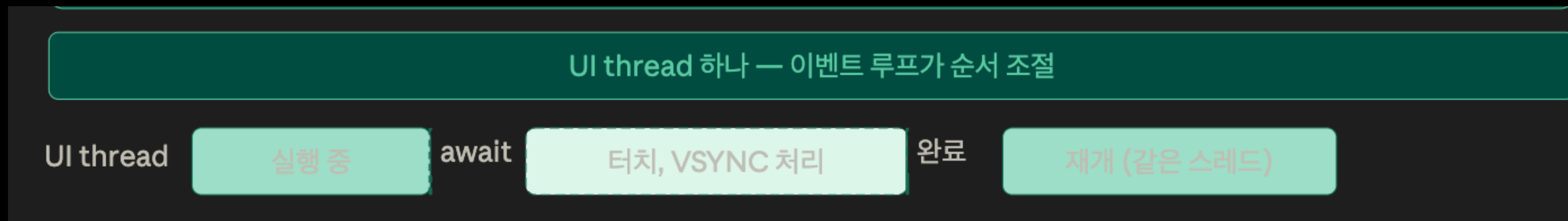
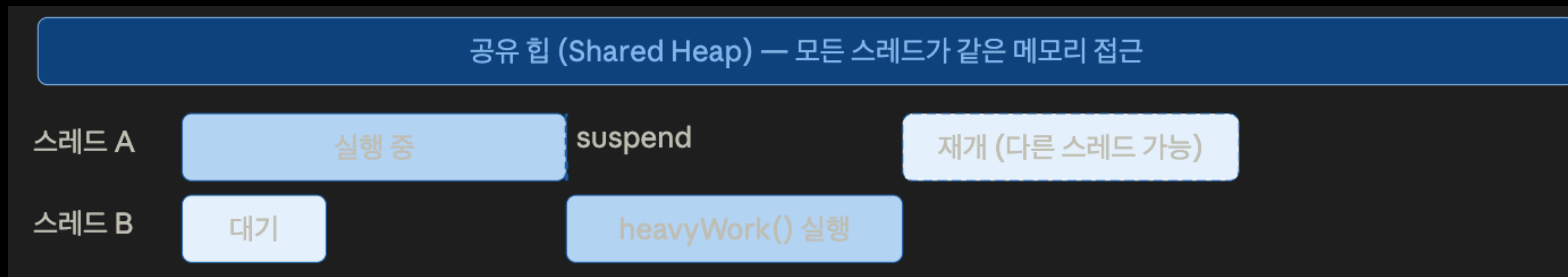
개발자가 관리

UI thread
Dart 코드, build(), 이벤트
루프

Isolate.run()
백그라운드 CPU 연산

= 같은 역할

= 같은 역할





Isolation 훔쳐보기



```
0
1  Future<void> sortAndShow() async {
2      // 무거운 연산 → Isolate.run()으로 보냄
3      // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4      // UI thread는 그 동안 터치, VSYNC 계속 처리
5      final sorted = await Isolate.run(() {
6          return items.sorted();
7      });
8
9      // 결과 받으면 같은 UI thread에서 재개
10     setState(() {
11         items = sorted;
12     });
13 }
```



```
0
1  Future<void> sortAndShow() async {
2      // 무거운 연산 → Isolate.run()으로 보냄
3      // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4      // UI thread는 그 동안 터치, VSYNC 계속 처리
5      final sorted = await Isolate.run(() {
6          return items.sorted();
7      });
8
9      // 결과 받으면 같은 UI thread에서 재개
10     setState(() {
11         items = sorted;
12     });
13 }
```

기존 Main 힙

Items [3,1,2]

Sorted = ???

Worker

Isolation 힙

비어있음



```
0
1 Future<void> sortAndShow() async {
2     // 무거운 연산 → Isolate.run()으로 보냄
3     // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4     // UI thread는 그 동안 터치, VSYNC 계속 처리
5     final sorted = await Isolate.run(() {
6         return items.sorted();
7     });
8
9     // 결과 받으면 같은 UI thread에서 재개
10    setState(() {
11        items = sorted;
12    });
13 }
```

기존 Main 힙

Items [3,1,2]

Sorted = ???



Byte 변환 -> 복사

Worker
Isolation 힙

비어있음



이벤트 처리중

Main에서는 다른 이벤트 처리

Worker에서는 연산

기존 Main 힙

터치 이벤트 처리
애니메이션 처리

Worker

Isolation 힙

Sorted 계산

```
0
1 Future<void> sortAndShow() async {
2     // 무거운 연산 → Isolate.run()으로 보냄
3     // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4     // UI thread는 그 동안 터치, VSYNC 계속 처리
5     final sorted = await Isolate.run(() {
6         return items.sorted();
7     });
8
9     // 결과 받으면 같은 UI thread에서 재개
10    setState(() {
11        items = sorted;
12    });
13 }
```



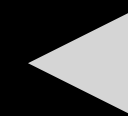
완료되었을때

```
0
1 Future<void> sortAndShow() async {
2     // 무거운 연산 → Isolate.run()으로 보냄
3     // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4     // UI thread는 그 동안 터치, VSYNC 계속 처리
5     final sorted = await Isolate.run(() {
6         return items.sorted();
7     });
8
9     // 결과 받으면 같은 UI thread에서 재개
10    setState(() {
11        items = sorted;
12    });
13 }
```

기존 Main 힙

Items [3,1,2]

Sorted = ???



SendPort/ReceivePort 발사!!

Worker

Isolation 힙

정렬 다했핑!!



```
0
1  Future<void> sortAndShow() async {
2      // 무거운 연산 → Isolate.run()으로 보냄
3      // items 직렬화 복사 → 새 스레드에서 정렬 → 결과 복사
4      // UI thread는 그 동안 터치, VSYNC 계속 처리
5      final sorted = await Isolate.run(() {
6          return items.sorted();
7      });
8
9      // 결과 받으면 같은 UI thread에서 재개
10     setState(() {
11         items = sorted;
12     });
13 }
```

기존 Main 힙

Items [3,1,2]

Sorted = ???

Worker

Isolation 힙

비어있음



Isolate.run() 실행 흐름

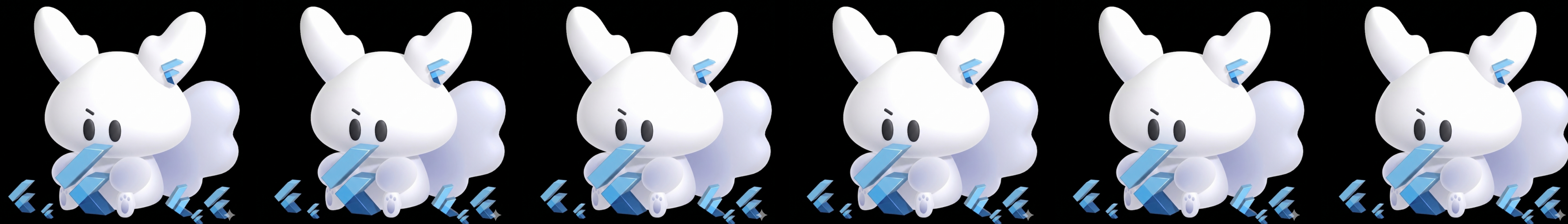
- Isolate.run() 호출
- items 직렬화 → Worker 힙으로 복사
- Worker 스레드에서 sorted() 실행 (별도 CPU 코어)
- Main은 이벤트 루프 유지 → UI 작업 계속 처리 (터치, VSYNC)
- 완료 시 sendPort.send(result)로 결과 전송
- result 직렬화 → Main 힙으로 복사
- 완료 이벤트 큐 등록 → UI 스레드 재개 → Worker Isolate



Mash-Up
iOS Team



그럼 멀티 스레드아님??





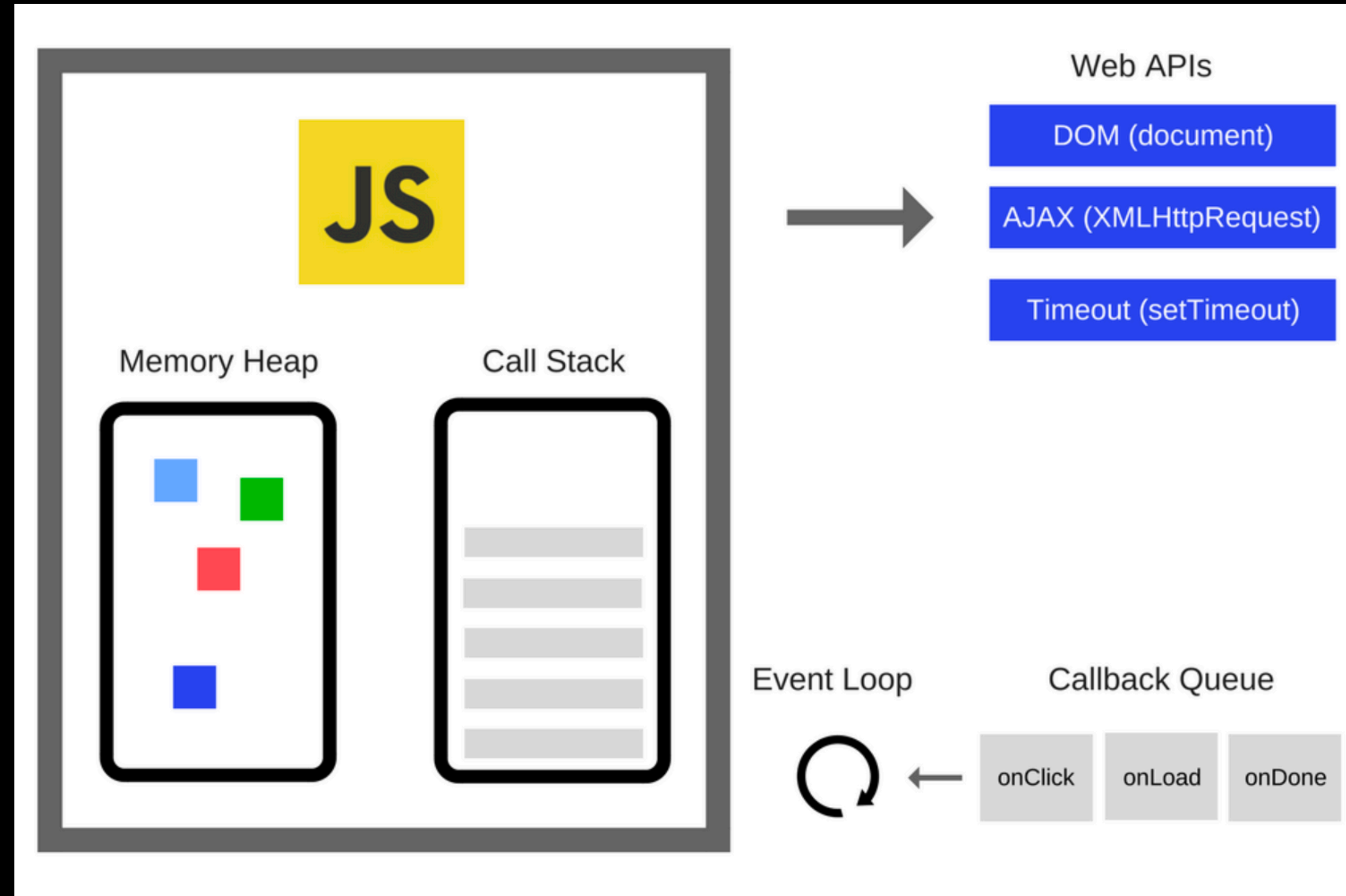
Mash-Up
iOS Team

Dart코드가 기본적으로 UIThread에서만 돌아서 아님ㅋㅋ



Mash-Up
iOS Team

왜 Javascript는 싱글스레드를 택했을까?





Mash-Up
iOS Team

상태관리



UIKit

- VC가 직접 상태 들고있음
- delegate 같은거 씬

SwiftUI

- @state 같은거 씬
- combine으로 스트림



1. Riverpod

```
1 // 상태 정의
2 final counterProvider = StateProvider<int>((ref) => 0);
3
4 // 위젯에서 사용
5 class CounterWidget extends ConsumerWidget {
6   @override
7   Widget build(BuildContext context, WidgetRef ref) {
8     final count = ref.watch(counterProvider);
9     return Text('$count');
10  }
11 }
12
13 // 상태 변경
14 ref.read(counterProvider.notifier).state++;
```



2. Bloc

```
1 // 이벤트 정의
2 abstract class CounterEvent {}
3 class Increment extends CounterEvent {}
4
5 // 상태 정의
6 class CounterState {
7   final int count;
8   CounterState(this.count);
9 }
10
11 // Bloc - 이벤트 받아서 새 상태 만들어냄
12 class CounterBloc extends Bloc<CounterEvent, CounterState> {
13   CounterBloc() : super(CounterState(0)) {
14     on<Increment>((event, emit) {
15       emit(CounterState(state.count + 1));
16     });
17   }
18 }
19
20 // 위젯에서 사용
21 BlocBuilder<CounterBloc, CounterState>(
22   builder: (context, state) => Text('${state.count}'),
23 );
```



Mash-Up
iOS Team

플러터, 해야하는가?



Mash-Up
iOS Team

플러터, 해야하는가?



Mash-Up
iOS Team





Ps
전자정부 표준프레임워크
신규버전
발표회



표준프레임워크센터
윤창원 이사

표준프레임워크 신규버전(v5.0) 소개

Chapter 1. 개요 및 추진 성과

표준프레임워크 주요 특징

Ministry of the Interior and Safety NIA NATIONAL INFORMATION SOCIETY AGENCY

개방형 표준 준수

오픈소스 기반으로 벤더 락인 방지,
기술 투명성과 생태계 확장성을 통해
다양한 개발자 협업 촉진

편리하고 다양한 환경 제공

Eclipse-VSCode 기반 IDE로 UML-ERD
모델링·개발 도구 지원, AI 코딩 에이전트
확장으로 코드 자동화·리뷰·문서화 효율화

모바일 환경 지원

모바일 웹·하이브리드 앱 내장으로
UX/UI 반영 개발 지원, 다양한
디바이스·OS 최적화로 사용자 접근성 향상

국가적 표준화 지향

정부·민간·학계 자문협의회를 통한
개발 방법론·아키텍처·인터페이스 표준화로
공공 시스템 일관성·품질 향상 및 호환성 문제 예방

상용 솔루션 연계

상용 솔루션 원활 연동 설계로 데이터·업무
호환성 보장, 유지관리 비용 절감 및
지속적 호환성 점검 체계 구축

변화 유연성

모듈화와 인터페이스 기반 연동으로
모듈 간 결합도 낮춤, 정책·기술 변화 시
최소 영향으로 유연 대응 가능

오픈소스 소프트웨어 사용현황

• Mobile Framework & Flutter Library

서비스 그룹	버전		라이선스	비고
Mobile Framework	Flutter	3.35.4	BSD-3-Clause	
	Dart	3.9.2	BSD-3-Clause	
공통	Http	1.5.0	BSD-3-Clause	
	Path Provider	2.12	BSD-3-Clause	
	Permission Handler	11.2.0	MIT	
	Device Info Plus	12.2.0	MIT	
	Get It	7.6.4	MIT	
	Shared Preferences	2.2.2	MIT	
	Flutter Secure Storage	9.2.4	BSD-3-Clause	
	Crypto	3.0.5	BSD-3-Clause	
	Flutter SVG	2.0.10	MIT	
	Url Launcher	6.2.5	BSD-3-Clause	
	Webview Flutter	4.4.2	BSD-3-Clause	
	Flutter Lints (Dev)	5.0.0	Apache 2.0	개발 환경일때만 사용



T-shaped Professional

Moderate ability in a broad set of skills
and deep ability in a core competency



Conventional Advice

Pi-shaped Professional

Moderate ability in a broad set of skills and
deep ability in two or more core competencies



Current Market Demand



Mash-Up
iOS Team

라이브 코딩



Mash-Up
iOS Team

크로스 플랫폼 하거나
네이티브 하거나



Mash-Up
iOS Team